R Programming Fundamentals

Vectors, Lists, Data Frames and dplyr

Ethiopian Statistics Service (ESS)

September 18 to 20, 2026

Learning Objectives

By the end of this lesson, you will be able to:

- Understand vectors in R
- Use `c()` to create vectors
- Identify common R data types
- Index and subset vectors
- Work with missing values
- Understand lists
- Create and manipulate data frames
- Filter rows and select columns
- Understand the pipe operator
- Use basic `dplyr` functions

Part 1

Vectors in R

What is a Vector?

A **vector** is a one-dimensional collection of values.

Example:

```
x <- c(10, 20, 30, 40)
```

The vector contains four values:

10 20 30 40

Important

A vector normally contains values of the same basic data type.

The Role of c()

The function c() means:

combine

It combines values into a vector.

```
x <- c(10, 20, 30, 40)
```

It can also combine text:

```
names <- c("Leta", "Sara", "Kebede")
```

Logical values:

```
passed <- c(TRUE, FALSE, TRUE)
```

Creating Vectors

```r
# Numeric vector
age <- c(16, 17, 18, 19)

# Character vector
names <- c("Leta", "Sara", "Kebede")

# Logical vector
passed <- c(TRUE, FALSE, TRUE)

# Integer vector
x <- c(1L, 2L, 3L)
```

Remember

The assignment operator <- stores the vector in an object.

Main Vector Types

Common basic types include:

Type	Example
Numeric	`c(10, 20, 30)`
Integer	`c(10L, 20L)`
Character	`c("A", "B")`
Logical	`c(TRUE, FALSE)`
Complex	`c(2+3i, 4+5i)`

Checking the Type of a Vector

Use `class()`:

```
x <- c(10, 20, 30)

class(x)
```

Use `typeof()`:

```
typeof(x)
```

Other useful functions:

```
is.numeric(x)
is.character(x)
is.logical(x)
is.integer(x)
```

Type Coercion

R tries to make elements compatible when different types are combined.
Example:

```
x <- c(10, 20, "30")
```

Because one element is character, R converts the numbers to character.

```
x
```

Result:

```
"10" "20" "30"
```

Warning

Be careful when mixing numeric and character values.

Part 2

Vector Indexing

What is Indexing?

Indexing means selecting elements from an object.
R starts counting positions from **1**.

```
scores <- c(45, 67, 82, 91, 55, 73)
```

Positions:

Position	1	2	3	4	5	6
Score	45	67	82	91	55	73

Selecting One Element

Use square brackets:

```
scores[3]
```

Result:

```
82
```

Therefore:

$$\texttt{scores[3]} = \text{third element}$$

Selecting Multiple Elements

Use `c()` inside brackets:

```
scores[c(2, 4, 6)]
```

Result:

```
67 91 73
```

You can also select a range:

```
scores[2:5]
```

Negative Indexing

Negative numbers mean:

exclude these positions

Example:

```
scores[-1]
```

Removes the first element.
To remove positions 2 and 4:

```
scores[-c(2,4)]
```

The `length()` Function

`length()` tells us how many elements are in a vector.

```
scores <- c(45, 67, 82, 91, 55, 73)

length(scores)
```

Result:

```
6
```

Naming Vector Elements

We can give names to vector elements.

```
scores <- c(75, 88, 92)

names(scores) <- c("Leta", "Sara", "Kebede")
```

Now:

```
scores["Sara"]
```

returns:

```
88
```

Part 3

Vector Operations

Basic Vector Functions

Suppose:

```
scores <- c(45, 67, 82, 91, 55, 73)
```

Useful functions:

```
sum(scores)
mean(scores)
min(scores)
max(scores)
median(scores)
sd(scores)
var(scores)
sort(scores)
```

Vectorized Operations

R can perform calculations on all elements at once.

```
x <- c(10, 20, 30)

x + 5
x * 2
x^2
```

Results:

```
15 25 35

20 40 60

100 400 900
```

Key idea

R performs the operation element by element.

Operations Between Vectors

```
x <- c(10, 20, 30)
y <- c(1, 2, 3)

x + y
```

Result:

```
11 22 33
```

Similarly:

```
x * y
```

Result:

```
10 40 90
```

The Recycling Rule

What happens when vectors have different lengths?

```
x <- c(10, 20, 30, 40)
y <- c(1, 2)

x + y
```

R recycles `y`:

$$10+1, \quad 20+2, \quad 30+1, \quad 40+2$$

Result:

```
11 22 31 42
```

Part 4

Conditions and Filtering

Logical Conditions

Suppose:

```
scores <- c(45, 67, 82, 91, 55, 73)
```

Test whether scores are greater than or equal to 70:

```
scores >= 70
```

Result:

```
FALSE FALSE TRUE TRUE FALSE TRUE
```

Filtering a Vector

We can use the logical condition to select values:

```
scores[scores >= 70]
```

Result:

```
82 91 73
```

Important

`vector[condition]` selects the values that satisfy the condition.

The `which()` Function

`which()` returns the **positions** where a condition is TRUE.

```
which(scores >= 70)
```

Result:

```
3 4 6
```

Compare:

```
scores[scores >= 70]
```

gives values:

```
82 91 73
```

ifelse()

`ifelse()` performs a vectorized condition.

```
ifelse(scores >= 70, "Pass", "Fail")
```

Result:

```
"Fail" "Fail" "Pass" "Pass" "Fail" "Pass"
```

Structure:

```
ifelse(condition, value_if_true, value_if_false)
```

Missing Values: NA

R uses NA for missing values.

```
x <- c(10, 20, NA, 40)
```

Check missing values:

```
is.na(x)
```

Result:

```
FALSE FALSE TRUE FALSE
```

Calculations with Missing Values

Consider:

```
x <- c(10, 20, NA, 40)
```

```
mean(x)
```

returns NA.
Use:

```
mean(x, na.rm = TRUE)
```

to ignore missing values.

Part 5

Lists

What is a List?

A **list** can contain different types of objects.
For example:

```
student <- list(
name = "Leta",
age = 16,
scores = c(75, 82, 91),
passed = TRUE
)
```

The list contains:

- Character
- Numeric
- Numeric vector
- Logical

Accessing List Elements with $

```
student$name
```

returns:

```
"Leta"
```

```
student$age
```

returns:

```
16
```

```
student$scores
```

returns:

```
75 82 91
```

List Indexing: []

Consider:

```
student["name"]
```

This returns a **list** containing the name.
But:

```
student[["name"]]
```

extracts the actual value.

Remember

[] selects; [[]] extracts.

Three Ways to Access Named Elements

```
student["name"]
student[["name"]]
student$name
```

Conceptually:

Syntax	Meaning
[]	Select
[[]]	Extract
$	Extract by name

Part 6

Data Frames

What is a Data Frame?

A data frame is one of the most important structures for data analysis in R.
It looks like a spreadsheet or database table.
Example:

```
students <- data.frame(
name = c("Leta", "Sara", "Kebede", "Hana"),
age = c(16, 15, 16, 15),
score = c(75, 88, 92, 81)
)
```

A Data Frame as a Table

Name	Age	Score
Leta	16	75
Sara	15	88
Kebede	16	92
Hana	15	81

Each column can represent a variable.
Each row represents an observation.

Selecting a Column

Using $:

```
students$score
```

Result:

```
75 88 92 81
```

We can also use:

```
students[["score"]]
```

Selecting Rows and Cells

First row:

```
students[1, ]
```

Second column:

```
students[, 2]
```

Third row:

```
students[3, ]
```

Second row, third column:

```
students[2, 3]
```

Result:

```
88
```

Data Frame Filtering

Select students with score ≥ 80:

```
students[students$score >= 80, ]
```

Result:

Name	Age	Score
Sara	15	88
Kebede	16	92
Hana	15	81

Part 7

Introduction to `dplyr`

What is dplyr?

`dplyr` is an R package designed for data manipulation.
Load it using:

```
library(dplyr)
```

Four important functions:

1. `select()` – choose columns
2. `filter()` – choose rows
3. `mutate()` – create/modify columns
4. `summarise()` – calculate summaries

`select()` – Choose Columns

To select only name and score:

```
students %>%
select(name, score)
```

Result:

Name	Score
Leta	75
Sara	88
Kebede	92
Hana	81

Remember

`select()` = columns

`filter()` – Choose Rows

Select students with score ≥ 85:

```
students %>%
filter(score >= 85)
```

Result:

Name	Age	Score
Sara	15	88
Kebede	16	92

Remember

`filter()` = rows

mutate() – Create a Variable

Create a new variable called status:

```
students %>%
mutate(
status = ifelse(
score >= 80,
"Pass",
"Fail"
)
)
```

Result of `mutate()`

Name	Age	Score	Status
Leta	16	75	Fail
Sara	15	88	Pass
Kebede	16	92	Pass
Hana	15	81	Pass

Remember

`mutate()` = create or modify columns

summarise()

Calculate the average score:

```
students %>%
summarise(
mean_score = mean(score)
)
```

Result:

```
82
```

We can calculate several statistics:

```
students %>%
summarise(
mean_score = mean(score),
highest = max(score),
lowest = min(score)
)
```

The Pipe Operator %>%

The pipe:

$$\%>\%$$

means approximately:

"Take the result and pass it to the next operation."

Example:

```
students %>%
filter(score >= 80) %>%
summarise(mean_score = mean(score))
```

Understanding the Pipeline

The following:

```
students %>%
filter(score >= 80) %>%
summarise(mean_score = mean(score))
```

means:

1. Start with `students`
2. Keep scores ≥ 80
3. Calculate the mean of the remaining scores

The result is:

```
87
```

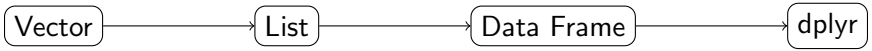
Part 8

Important Summary

Key Functions to Remember

Function	Purpose
`c()`	Combine values
`length()`	Count elements
`names()`	Name elements
`sum()`	Total
`mean()`	Average
`min()`	Minimum
`max()`	Maximum
`which()`	Find positions
`ifelse()`	Conditional values
`select()`	Choose columns
`filter()`	Choose rows
`mutate()`	Create/modify columns
`summarise()`	Calculate summaries

The Big Picture



- Vector: collection of values
- List: collection of different objects
- Data frame: table of observations and variables
- `dplyr`: powerful data manipulation

Practice Exercises

Exercise 1: Vectors

Create:

```
scores <- c(45, 67, 82, 91, 55, 73)
```

Answer:

1. What is `scores[3]`?
2. What is `scores[c(2,4,6)]`?
3. Select scores ≥ 70.
4. Find the positions of scores ≥ 70.
5. Calculate the mean score.

Exercise 2: Data Frames

Use:

```
students <- data.frame(
name = c("Abate", "Sara", "Kebede", "Hana"),
age = c(16, 15, 16, 15),
score = c(75, 88, 92, 81)
)
```

Answer:

1. Select the `name` column.
2. Select the third row.
3. Select row 2, column 3.
4. Select students with score ≥ 85.
5. Select only the `score` column.

Exercise 3: `dplyr`

Using the same data:

1. Use `select()` to choose `name` and `score`.
2. Use `filter()` for score ≥ 85.
3. Use `mutate()` to create `status`.
4. Calculate the average score using `summarise()`.
5. Find the highest score.

Challenge Exercise

Write one pipeline that:

1. Keeps students with score ≥ 80;
2. Creates a variable called `result`;
3. Gives "Excellent" when score ≥ 90;
4. Gives "Pass" otherwise;
5. Selects only `name`, `score`, and `result`.

Challenge Exercise

Write one pipeline that:

1. Keeps students with score ≥ 80;
2. Creates a variable called `result`;
3. Gives "Excellent" when score ≥ 90;
4. Gives "Pass" otherwise;
5. Selects only `name`, `score`, and `result`.

Possible structure:

```
students %>%
filter(...) %>%
mutate(...) %>%
select(...)
```

Next Lesson

Next we will study:

`group_by()` and grouped analysis

We will learn how to calculate statistics by groups such as:

- Region
- Gender
- Location
- ISIC sector
- Category

Example:

```
data %>%
group_by(region) %>%
summarise(
mean_sales = mean(sales, na.rm = TRUE)
)
```

Thank You!

Questions and Practice